DDM501 · INDIVIDUAL ASSIGNMENT 2 · ML PIPELINE DESIGN & MLOPS ANALYSIS

ML Pipeline and MLOps Design for a Student Dropout Early Warning System

Student	**Nguyen Thi Hong Hanh**
Student ID	**25MS13316**
Continues	Assignment 1 – ML System Design Document (dropout early warning)
Experimental data	UCI "Predict Students' Dropout and Academic Success" (4,424 students) as a proxy
Stack	Python 3.11 · scikit-learn · XGBoost · pandera · MLflow 2.17 · Apache Airflow 2.10 · Docker

> All code listings are excerpts of a working prototype, and all numbers come from actual runs. Institutional figures for BTEC FPT British College remain the assumptions stated in Assignment 1.

0. Link to Assignment 1

Problem statement (recap). Every week, score the dropout risk of every enrolled student at BTEC FPT British College (assumed 12,000 students, three 15-week semesters per year) at least 8 weeks before the semester ends. **Tier A** (top 10 % risk) goes to academic advisors for personal counselling; **Tier B** (top 10–25 %) goes to the student care team for check-in calls. Business target: reduce annual dropout from 8 % to 6.5 %.

Architecture (recap). Two pipelines share one feature module: a **training pipeline** runs each semester or on drift; a **weekly batch scoring pipeline** runs every Sunday. Components: nightly extraction → data validation → pseudonymised warehouse → feature computation → training with MLflow tracking → quality gate + human approval → model registry → batch scoring → advisor/care-team web app; monitoring for pipeline health, drift and fairness; fallback rule B2 when data is invalid.

Refinements after Sessions 3–5.

- **Proxy dataset.** No real college data is available, so experiments use the public UCI dataset (4,424 students, 36 attributes, 32.1 % dropout). It has no weekly features, so the feature groups "enrolment / semester 1 / semester 2" simulate predicting earlier or later.
- **Split strategy.** The dataset has no enrolment date, so a stratified split replaces the time-based split of Assignment 1; production data will return to a semester-based split.
- **Selection rule as code.** Trade-off 5.1 of Assignment 1 ("choose gradient boosting only if its PR-AUC beats logistic regression by ≥ 0.02") is now the function `select_model`, parameterised in `params.yaml`.
- **Additions.** Cross-validated PR-AUC with standard deviation to separate real differences from noise; a `previous-champion` alias for one-step rollback; PSI drift detection inside the scoring DAG.

1. Pipeline Design 25%

1.1 Pipeline architecture diagram

1 Data ingestion

Nightly export from SIS, LMS, attendance, finance and care-team call log is copied into an immutable, dated snapshot; its SHA-256 is recorded.

Output: raw CSV snapshot + data_sha256 | Gate: file readable, required columns present

2 Data validation

pandera schema (label values, age 15–80, grades 0–20, binary flags), missing-value ratio, minimum row count.

Output: validated DataFrame | Gate: missing ≤ 5 %, rows ≥ 1,000, all checks pass | fail → stop, alert, fallback rule B2

▼

3 Preprocessing

Binary label (Dropout = 1); stratified 60/20/20 train/validation/test split with fixed seed.

Output: train 2,654 · val 885 · test 885 | Gate: equal dropout rate in all splits; no overlap

▼

4 Feature engineering

Select feature groups, drop sensitive attributes, add approval rates, grade trend and financial-risk score; one-hot encode and scale.

Output: feature matrix (inside sklearn pipeline) | Gate: no sensitive column; same code for training and scoring

▼

5 Model training

Twelve configurations E01–E12, each an MLflow run; randomized hyperparameter search with 5-fold CV.

Output: fitted pipelines + MLflow runs | Gate: params, metrics, model and feature list logged

▼

6 Model evaluation

PR-AUC, CV PR-AUC ± SD, Recall/Precision@k, Brier, ECE, fairness gap; selection rule and quality gate.

Output: results table + selected model | Gate: PR-AUC ≥ 0.80; fairness gap ≤ 0.10; ≤ 0.01 worse than champion | fail → keep current champion

▼

7 Deployment

Register version with alias challenger; human approval moves alias champion; weekly DAG scores with champion and assigns tiers.

Output: model version + Tier A/B lists | Gate: human approval; previous-champion kept for rollback

Figure 1. Training pipeline: seven stages, the artefact passed between them, and the quality gate each must pass. The weekly scoring pipeline reuses stages 1, 2 and 4 with identical code, then scores with the champion model (Figure 3).

1.2 Stage specifications

Stage	Purpose	Input → output	Key operations (module)	Quality gate
1. Ingestion	Fixed, traceable copy of the data	Source exports → dated CSV snapshot + SHA-256	`data.ingest`, `file_sha256`; normalise column names (the export contains a trailing tab)	File readable, required columns present
2. Validation	Stop corrupted data before training or scoring	Raw DataFrame → validated DataFrame	`data.validate`: pandera schema, value ranges, missing ratio, row count	Raises `DataValidationError` → pipeline stops; scoring DAG branches to rule B2
3. Preprocessing	Create the label and honest evaluation sets	DataFrame → train 2,654 / val 885 / test 885	`make_label`; stratified `split` with `random_state=42`	Dropout rate 32.1 % in each split; disjoint indices (unit-tested)

Stage	Purpose	Input → output	Key operations (module)	Quality gate
4. Feature engineering	Turn raw records into risk signals	Raw columns → feature matrix	`select_columns` (drops Gender, Nacionality, Marital status, International); `EngineeredFeatures` (approval rates, grade trend, financial risk); `ColumnTransformer` (one-hot, scaling)	No sensitive attribute (unit-tested); divide-by-zero safe
5. Training	Systematically compare configurations	Feature matrix → fitted pipelines	`train.run_experiment`: one configuration = one MLflow run; `RandomizedSearchCV` (25 candidates)	Run must log params, metrics, model, feature list
6. Evaluation	Select by operational metrics	Validation predictions → metrics + selected model	`evaluate`, `select_model`, `passes_quality_gate`	PR-AUC ≥ 0.80; gender recall gap ≤ 0.10; ≤ 0.01 below champion
7. Deployment	Controlled release	Run → model version → Tier A/B lists	`register_candidate` (challenger), `promote` (champion, keeps previous-champion), `score_with_champion`, `assign_and_publish`	Human approval required; test set used once, for the selected model only

src/dropout/data.py – validation quality gate

```
def validate(df: pd.DataFrame, params: dict[str, Any]) -> pd.DataFrame:
    """Stage 2 - quality gate: schema, volume and completeness checks.

    Raises:
        DataValidationError: if any check fails, so no model is trained or
            scored on corrupted data.
    """
    cfg = params["data"]
    if len(df) < cfg["min_rows"]:
        raise DataValidationError(f"Only {len(df)} rows, expected >= {cfg['min_rows']}")
    missing = float(df.isna().mean().max())
    if missing > cfg["max_missing_ratio"]:
        raise DataValidationError(f"Missing ratio {missing:.2%} above threshold")
    try:
        df = build_schema(params).validate(df, lazy=True)
    except pa.errors.SchemaErrors as exc:
        raise DataValidationError(str(exc.failure_cases.head())) from exc
    logger.info("Validation passed (max missing ratio %.2f%%)", missing * 100)
```

src/dropout/features.py – shared feature engineering

```
class EngineeredFeatures(BaseEstimator, TransformerMixin):
    """Add ratio and trend features derived from semester results.

    Only features whose source columns are present are created, so the same
    transformer works for "enrollment only" and "full" feature sets.
    """

    def fit(self, X: pd.DataFrame, y: pd.Series | None = None) -> EngineeredFeatures:
        """Stateless transformer; nothing to learn."""
        return self

    def transform(self, X: pd.DataFrame) -> pd.DataFrame:
        """Return a copy of ``X`` with engineered columns appended."""
        out = X.copy()
        for sem, tag in (("1st", "s1"), ("2nd", "s2")):
            enrolled = f"Curricular units {sem} sem (enrolled)"
```

```
            approved = f"Curricular units {sem} sem (approved)"
            if enrolled in out and approved in out:
                # Share of enrolled modules passed; 0 when nothing was enrolled.
                out[f"approval_rate_{tag}"] = np.where(
                    out[enrolled] > 0, out[approved] / out[enrolled].clip(lower=1), 0.0
                )
        grade1, grade2 = "Curricular units 1st sem (grade)", "Curricular units 2nd sem (grade)"
        if grade1 in out and grade2 in out:
            out["grade_trend"] = out[grade2] - out[grade1]
        if "Debtor" in out and "Tuition fees up to date" in out:
            out["financial_risk"] = out["Debtor"] + (1 - out["Tuition fees up to date"])
        return out
```

1.3 Design rationale

- **Structure.** Each stage has one responsibility and an explicit input/output artefact, so failures are localised and a single stage can be re-run. Validation comes first because bad data that slips through silently corrupts every later stage.
- **Modularity and reuse.** Every stage is a plain, typed function in the `dropout` package that exchanges file paths or objects and has no Airflow dependency; the same functions are called by the experiment script, the DAGs and the unit tests. `EngineeredFeatures` lives *inside* the scikit-learn pipeline stored in MLflow, so scoring cannot compute features differently from training (no training-serving skew). A new algorithm is one branch in `build_estimator`; a new experiment is one line in `MATRIX`.
- **Error handling and recovery.** (1) Invalid data → stop and score with fallback rule B2 so advisors still receive a list; (2) transient errors → Airflow retries twice with exponential back-off; (3) candidate fails the gate → keep the current champion and log the reasons; (4) a bad model in production → move the champion alias back to `previous-champion` without rebuilding; (5) dated snapshots make every task idempotent and re-runnable.
- **Scalability.** Scoring 4,424 students takes under one second and grows linearly; for 12,000–50,000 students scoring can be partitioned by programme and run in parallel. The most expensive step, hyperparameter search (21 s for E07), runs once per semester and can be scaled with `n_jobs` or Optuna. Switching `MLFLOW_TRACKING_URI` from SQLite to PostgreSQL + MinIO supports multiple users with no code change.

2. Experiment Tracking & Metrics Analysis 25%

2.1 Experiment design

- **Experiment variables:** *algorithm* (rule, logistic regression, random forest, XGBoost); *hyperparameters* (defaults vs. randomized search over n_estimators, max_depth, learning_rate, subsample, colsample_bytree, min_child_weight); *features* (enrolment, semester 1, semester 2, financial groups); *data preprocessing* (no rebalancing, class weights, SMOTE, isotonic calibration).
- **Baselines:** E01 is rule B2 (the production fallback rule from Assignment 1); E02 is logistic regression (baseline B3). A more complex model must beat E02 by ≥ 0.02 PR-AUC to be selected.
- **Fair comparison:** identical split and seed (42) for all runs; selection on the validation set; the test set is used **once**, for the final model.

ID	Algorithm	Hyperparameters	Features	Rebalancing	Question answered
E01	Rule B2	debt + unpaid fees + no module passed in sem. 1	Full	–	Non-ML baseline
E02	Logistic regression	max_iter=2000	Full	None	Baseline B3, interpretable
E03	Logistic regression	class_weight=balanced	Full	Class weights	Does reweighting help?
E04	Random forest	400 trees, min_samples_leaf=3	Full	Class weights	Bagging ensemble
E05	XGBoost	defaults	Full	None	Untuned boosting
E06	XGBoost	scale_pos_weight ≈ 2.1	Full	Class weights	Effect on calibration
E07	XGBoost	random search, 25 candidates, 5-fold CV	Full	None	Hyperparameter tuning
E08	XGBoost (E07 params)	as E07	Full	SMOTE	Synthetic oversampling
E09	XGBoost (E07 params)	+ isotonic calibration, cv=5	Full	None	Probability calibration
E10	XGBoost (E07 params)	as E07	Enrolment + financial	None	Earliest possible prediction
E11	XGBoost (E07 params)	as E07	+ Semester 1	None	Early prediction
E12	XGBoost (E07 params)	as E07	No financial features	None	Privacy trade-off

`src/dropout/experiments.py – experiment matrix as code`

```
MATRIX: list[ExperimentConfig] = [
    ExperimentConfig("E01", "Baseline B2: extended rule", "rule"),
    ExperimentConfig("E02", "Baseline B3: logistic regression", "logreg"),
    ExperimentConfig("E03", "Logistic regression, class_weight", "logreg", balance="class_weight"),
    ExperimentConfig(
        "E04",
        "Random forest, class_weight",
        "rf",
        balance="class_weight",
        hyperparams={"min_samples_leaf": 3},
    ),
    ExperimentConfig("E05", "XGBoost default", "xgb"),
    ExperimentConfig("E06", "XGBoost, scale_pos_weight", "xgb", balance="class_weight"),
    ExperimentConfig("E07", "XGBoost tuned (random search, 5-fold CV)", "xgb", tune=True),
    ExperimentConfig("E08", "XGBoost tuned + SMOTE", "xgb", balance="smote"),
```

```
    ExperimentConfig("E09", "XGBoost tuned + isotonic calibration", "xgb", calibrate=True),
    ExperimentConfig(
        "E10",
        "XGBoost tuned, enrollment data only (earliest)",
        "xgb",
        feature_groups=["enrollment", "financial"],
    ),
    ExperimentConfig(
        "E11",
        "XGBoost tuned, up to semester 1 (early)",
        "xgb",
        feature_groups=["enrollment", "financial", "semester1"],
    ),
    ExperimentConfig(
        "E12",
        "XGBoost tuned, no financial features (privacy)",
        "xgb",
        feature_groups=["enrollment", "semester1", "semester2"],
    ),
]
DEPENDS_ON_TUNED = {"E08", "E09", "E10", "E11", "E12"}
```

2.2 Metrics strategy

Type	Metric	Rationale
Primary	PR-AUC (validation) and 5-fold CV PR-AUC ± SD	Measures how well at-risk students are ranked at the top, focusing on the minority class; the CV standard deviation shows whether a difference exceeds noise. Used for model selection.
Primary (operational)	Recall@10 %, Recall@25 %	Correspond to Tier A (advisors) and Tiers A+B (plus care team): the share of all dropouts reached within contact capacity.
Secondary	Precision@10 %, ROC-AUC	Whether the advisors' list is accurate; ROC-AUC for comparison with literature.
Secondary	Brier score, ECE	The dashboard sums probabilities to estimate expected dropouts per programme, so probabilities must be calibrated.
Constraint	Recall@25 % gap by gender	Fairness condition from Assignment 1; a gap above 0.10 makes a model ineligible.
Secondary	Training time	Operational cost per retraining.

Business–model alignment. Following the goal hierarchy of Assignment 1: Recall@10 % × number of dropouts = at-risk students met by advisors; × assumed 30 % retention = students kept; Recall@25 % adds the care-team tier. Note that the proxy data has a 32 % base rate, so a 10 % list can contain at most 31 % of all dropouts: Recall@10 % is capped near 0.31 even though the list is almost perfectly precise. At the assumed BTEC rate of 2.7 % per semester the constraint flips (the list has room for every case, but precision will be much lower). Metric definitions carry over unchanged; only the values differ.

2.3 Code: MLflow setup, logging and model registration

`.env.example – infrastructure settings as environment variables`

```
MLFLOW_TRACKING_URI=sqlite:///mlflow.db
MLFLOW_ARTIFACT_ROOT=./mlartifacts
DROPOUT_PARAMS_PATH=params.yaml
DROPOUT_DATA_PATH=data/raw/students.csv
ALERT_EMAIL=data-team@example.edu
```

`src/dropout/experiments.py – MLflow experiment setup`

```
def main() -> pd.DataFrame:
    """Run every experiment, then gate, register and promote the best model."""
    params = load_params()
    set_global_seed(params["project"]["seed"])
    mlflow.set_tracking_uri(load_settings().tracking_uri)
    mlflow.set_experiment(params["mlflow"]["experiment_name"])

    splits = load_splits(params, PROJECT_ROOT / params["data"]["raw_path"])
```

src/dropout/train.py – parameter, metric and artifact logging

```
    with mlflow.start_run(run_name=f"{cfg.exp_id}-{cfg.algorithm}") as run:
        mlflow.set_tags(
            {"exp_id": cfg.exp_id, "description": cfg.description, "data_sha256": splits.data_hash}
        )
        mlflow.log_params(
            {
                "algorithm": cfg.algorithm,
                "feature_groups": ",".join(cfg.feature_groups),
                "n_features_raw": len(columns),
                "balance": cfg.balance,
                "tune": cfg.tune,
                "calibrate": cfg.calibrate,
                "seed": seed,
                **cfg.hyperparams,
            }
        )
        start = time.perf_counter()
        ...  # build, (tune), cross-validate and fit the pipeline
        mlflow.log_metrics(metrics)
        mlflow.log_dict({"columns": columns}, "feature_columns.json")
        if model is not None:
            mlflow.sklearn.log_model(
                model,
                artifact_path="model",
                signature=infer_signature(X_val.head(5), scores[:5]),
                input_example=X_val.head(3),
```

src/dropout/registry.py – registration, versioning and aliases

```
def register_candidate(run_id: str, params: dict[str, Any], tags: dict[str, str]) -> str:
    """Register the run's model as a new version with alias ``challenger``.

    Returns:
        str: the new model version number.
    """
    name = params["mlflow"]["registered_model"]
    version = mlflow.register_model(f"runs:/{run_id}/model", name)
    client = MlflowClient()
    for key, value in tags.items():
        client.set_model_version_tag(name, version.version, key, value)
    client.set_registered_model_alias(name, "challenger", version.version)
    logger.info("Registered %s v%s as challenger", name, version.version)
    return str(version.version)


def promote(params: dict[str, Any], version: str, approved_by: str) -> None:
    """Move the ``champion`` alias after human approval (keeps history for rollback)."""
    name = params["mlflow"]["registered_model"]
    client = MlflowClient()
    try:
        previous = client.get_model_version_by_alias(name, "champion").version
        client.set_registered_model_alias(name, "previous-champion", previous)
    except MlflowException:
        pass  # first deployment: nothing to keep
    client.set_registered_model_alias(name, "champion", version)
```

```
    client.set_model_version_tag(name, version, "approved_by", approved_by)
    logger.info("Promoted %s v%s to champion (approved by %s)", name, version, approved_by)
```

Each model version is tagged with `exp_id`, `git_commit`, `data_sha256` and `approved_by`, so any version traces back to the exact code and data. Actual registry state after the run: `dropout-risk` version 1 with aliases `champion` and `challenger`, git_commit `bbc0168`, data_sha256 `19737f02127e`; 12 runs in experiment `dropout-early-warning`.

2.4 Experiment results (validation set, n = 885, 284 dropouts)

ID	PR-AUC	CV PR-AUC ± SD	ROC-AUC	Rec@10%	Prec@10%	Rec@25%	Brier	ECE	Gender gap	Sec.
E01	0.728	–	0.837	0.275	0.886	0.613	–	–	0.042	0.00
E02	0.874	0.871 ± 0.017	0.924	0.296	0.955	0.694	0.092	0.023	0.077	0.20
E03	0.874	0.869 ± 0.018	0.925	0.296	0.955	0.690	0.102	0.083	0.099	0.27
E04	0.875	0.870 ± 0.014	0.924	0.299	0.966	0.683	0.098	0.052	**0.113**	6.23
E05	0.871	0.870 ± 0.012	0.918	0.296	0.955	0.669	0.103	0.069	0.042	0.62
E06	0.875	0.867 ± 0.011	0.919	0.303	0.977	0.676	0.101	0.060	0.070	0.62
E07	0.877	0.881 ± 0.016	0.925	0.299	0.966	0.687	0.092	0.020	0.091	21.27
E08	0.874	0.882 ± 0.016	0.925	0.299	0.966	0.690	0.093	0.020	0.099	2.36
E09	0.878	0.877 ± 0.016	0.925	0.296	0.955	0.680	0.094	0.042	0.063	5.61
E10	0.737	0.737 ± 0.023	0.822	0.292	0.943	0.542	0.149	0.047	0.042	0.91
E11	0.855	0.852 ± 0.018	0.903	0.303	0.977	0.672	0.106	0.029	0.091	1.02
E12	0.842	0.860 ± 0.015	0.906	0.289	0.932	0.669	0.106	0.030	**0.127**	1.27

Green row: selected model. Red: fairness gap above 0.10 (ineligible). Maximum achievable Recall@10 % is 0.31 (88 list slots / 284 dropouts). A complete re-run from an empty tracking store reproduced every metric exactly.

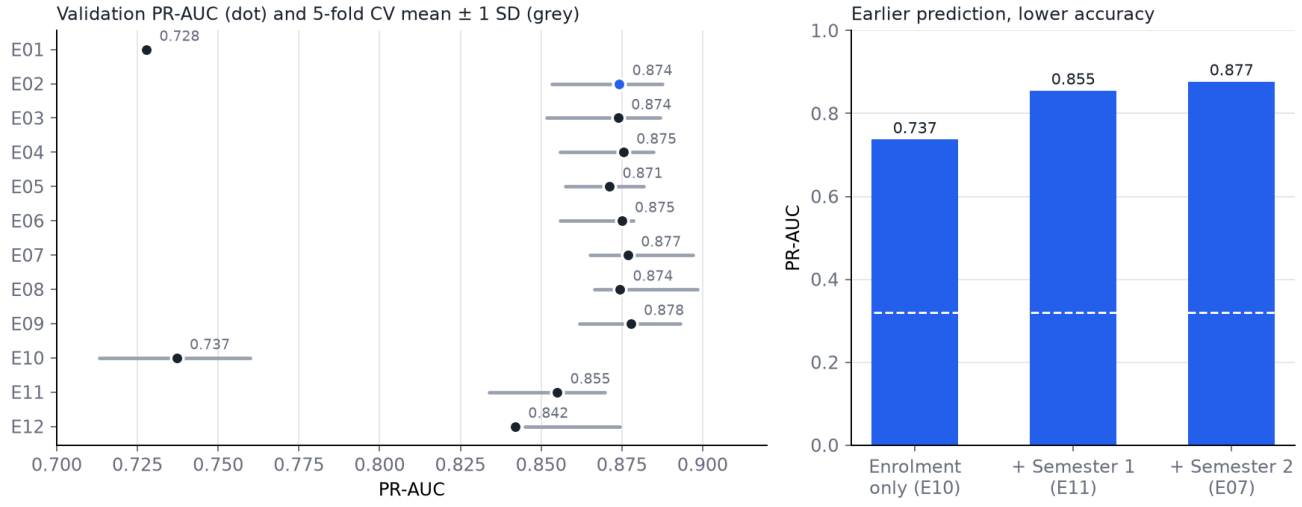


Figure 2. Left: validation PR-AUC (dots) and 5-fold CV mean ± 1 SD (grey bars); E02 highlighted. Right: PR-AUC by data available at prediction time (dashed line = random, 0.32).

2.5 Analysis of patterns and trade-offs

- **ML clearly beats the rule.** E01 reaches PR-AUC 0.728 while ML models with full features reach 0.871–0.878; Recall@25 % rises from 0.613 to about 0.69. This confirms the "why ML" argument of Assignment 1.

- **Among ML models, differences are within noise.** E02–E09 differ by at most 0.007 validation PR-AUC, while CV standard deviations are 0.011–0.018. The best validation model, E09 (0.878), beats logistic regression E02 by only 0.0036. On CV, tuned XGBoost (E07, 0.881) leads E02 (0.871) by about 0.01, still less than one SD. With small tabular data and near-linear academic signals, complex models have little room to win.
- **Ranking vs. calibration.** Class weighting does not improve PR-AUC but damages probabilities: ECE rises from 0.023 (E02) to 0.083 (E03); XGBoost with `scale_pos_weight` (E06) has ECE 0.060. Top-k lists only need ranking, so reweighting brings cost without benefit.
- **SMOTE and isotonic calibration do not pay off.** E08 has the highest CV PR-AUC (0.882) but no validation gain and adds synthetic data. E09 has ECE 0.042, *worse* than uncalibrated E07 (0.020): XGBoost with log-loss is already well calibrated and isotonic fitting on ~530-sample folds is noisy. Calibration should be added only when measured to help.
- **Early detection vs. accuracy** (trade-off 5.5 of Assignment 1). Enrolment data only (E10): PR-AUC 0.737, close to the rule; with semester 1 (E11): 0.855; with semester 2 (E07): 0.877. Most signal arrives with the **first academic results**, supporting weekly scoring whose confidence grows once interim assessments exist.
- **Privacy vs. accuracy** (trade-off 5.4). Removing the two financial features (E12) lowers PR-AUC by 0.035 and raises the fairness gap to 0.127, above the limit. Fee status is an important, legitimate signal: keep it, but protect it (pseudonymisation, role-based access, no amounts shown to callers).
- **Fairness.** Random forest (E04, gap 0.113) and E12 exceed 0.10 and are excluded automatically, although E04 has a competitive PR-AUC; ranking by PR-AUC alone would have hidden this.
- **Cost.** E07 needs 21.3 s for tuning versus 0.20 s for E02; at this scale cost does not decide.

src/dropout/experiments.py – model selection rule

```
def select_model(results: list[ExperimentResult], params: dict) -> ExperimentResult:
    """Pick the production candidate.

    Rules: (1) only ML models using the full feature set that pass the
    fairness limit are eligible; (2) the top PR-AUC model is chosen only if
    it beats the simple baseline by ``min_gain_over_baseline``; otherwise the
    simpler, explainable baseline is kept.
    """
    sel = params["selection"]
    max_gap = params["quality_gate"]["max_fairness_gap"]
    eligible = [
        r
        for r in results
        if r.model is not None
        and r.config.feature_groups == FULL
        and r.val_metrics["fairness_gap_at_25"] <= max_gap
    ]
    top = max(eligible, key=lambda r: r.val_metrics["pr_auc"])
    simple = next(r for r in results if r.config.exp_id == sel["simple_baseline"])
    gain = top.val_metrics["pr_auc"] - simple.val_metrics["pr_auc"]
    logger.info("Top model %s, gain over %s = %.4f", top.config.exp_id, simple.config.exp_id, gain)
    return top if gain >= sel["min_gain_over_baseline"] else simple
```

2.6 Recommendation

Select E02: logistic regression, no rebalancing, full feature set. The best PR-AUC model (E09) is only 0.0036 ahead, far below the 0.02 margin set in Assignment 1. E02 is directly interpretable for advisors (coefficients show which factors raise risk), well calibrated (ECE 0.023), within the fairness limit (gap 0.077) and trains instantly. `select_model` reached this decision automatically. Results on the untouched test set (885 students):

0.900 PR-AUC	**0.989** Precision@10 % (Tier A)	**0.704** Recall@25 % (Tiers A+B)	**0.025** ECE	**0.073** Gender recall gap

Test performance matches validation, so there is no sign of overfitting to the validation set. The runner-up is E07 (XGBoost; best parameters: subsample=0.8, n_estimators=500, min_child_weight=3, max_depth=3, learning_rate=0.05, colsample_bytree=0.9). It should be re-evaluated when real weekly features exist, because non-linear interactions (e.g. rising absence *and* overdue fees) may let it clear the 0.02 margin.

3. Workflow Orchestration Design 20%

3.1 DAG architecture

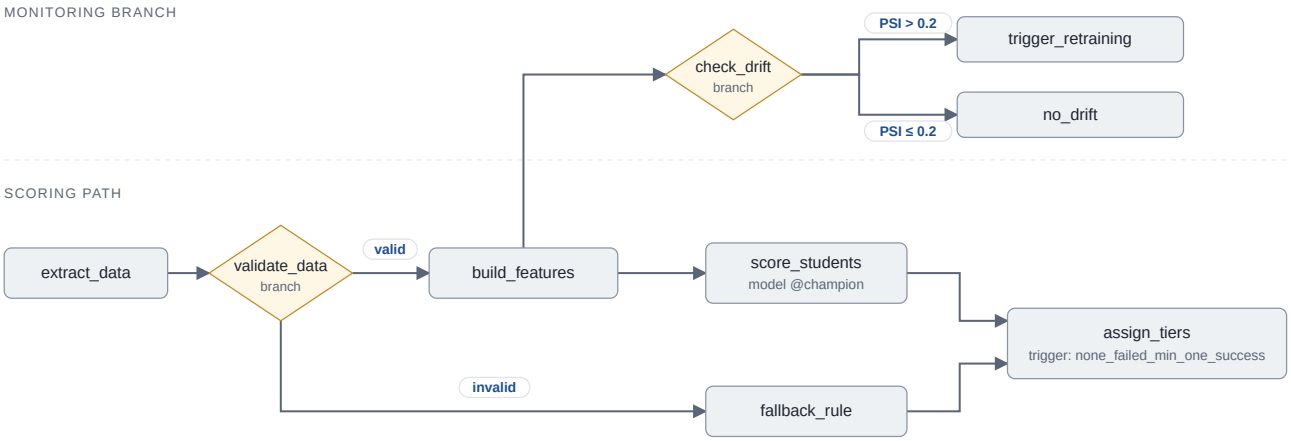


Figure 3. DAG `dropout_weekly_scoring` (Sunday 02:00, Asia/Ho_Chi_Minh).

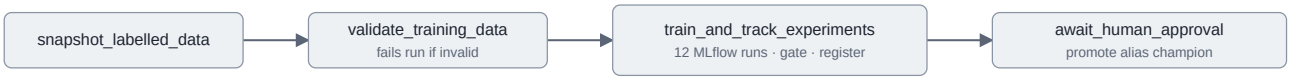


Figure 4. DAG `dropout_training` (after each semester's labels arrive, or triggered by drift). Both DAGs load in Apache Airflow 2.10.3 with no import errors (9 and 4 tasks).

Task	Description	Dependencies
`extract_data`	Copy the nightly export into a dated snapshot; return its path via XCom	None (first task)
`validate_data`	Branch: valid → `build_features`; invalid → `fallback_rule`	Needs the snapshot from extract_data
`build_features`	Compute features with the shared module; write file	Runs only if data is valid
`score_students`	Load `models:/dropout-risk@champion`; score all students	Needs the feature file
`fallback_rule`	Score with rule B2 so a list is still produced	Runs only if data is invalid
`assign_tiers`	Rank, assign Tier A (advisor) / Tier B (care team), publish lists	After *either* scoring branch succeeds (`NONE_FAILED_MIN_ONE_SUCCESS`)
`check_drift`	Branch: max feature PSI > 0.2 → retrain	Needs the feature file; parallel to scoring
`trigger_retraining / no_drift`	Trigger the training DAG, or end	Outcome of check_drift
`snapshot_labelled_data`	Snapshot data whose labels are complete	None (training DAG start)
`validate_training_data`	Fail the run if validation fails (current champion is kept)	Needs the labelled snapshot
`train_and_track_experiments`	Run E01–E12, log to MLflow, gate, register challenger	Needs validated data
`await_human_approval`	Explicit stop: a reviewer inspects MLflow and moves the champion alias	After training

3.2 Scheduling strategy

DAG	Trigger type	Frequency	Reason
dropout_weekly_scoring	Time-based (cron `0 2 * * 0`)	Weekly, Sunday 02:00; `catchup=False`, `max_active_runs=1`	Advisors work weekly; lists must be ready before Monday 06:00 (NFR1, 4-hour SLA)
dropout_training	Time-based + event-based (TriggerDagRunOperator on drift) + manual	Three times a year (`0 3 1 1,5,9 *`), after each semester's labels	Labels only exist after each semester; drift should not wait for the next one

3.3 Code: DAG definitions, task definitions and dependencies

dags/dropout_training_dag.py – alerting callback and default arguments

```
def notify_failure(context: dict[str, Any]) -> None:
    """Alerting: e-mail the data team with the failed task and log link."""
    ti = context["task_instance"]
    send_email(
        to=ALERT_EMAIL,
        subject=f"[dropout] {ti.dag_id}.{ti.task_id} failed",
        html_content=f"Run {context['run_id']} failed. Log: {ti.log_url}",
    )


DEFAULT_ARGS: dict[str, Any] = {
    "owner": "data-team",
    "retries": 2,  # transient DB/network errors
    "retry_delay": timedelta(minutes=10),
    "retry_exponential_backoff": True,
    "execution_timeout": timedelta(hours=1),
    "on_failure_callback": notify_failure,
}
```

dags/dropout_training_dag.py – task definitions (thin wrappers around the package)

```
def validate_data(**ctx: Any) -> str:
    """Branch: continue with ML scoring if data is valid, else use the rule."""
    from dropout.pipeline_tasks import validate_snapshot

    snapshot = ctx["ti"].xcom_pull(task_ids="extract_data")
    return "build_features" if validate_snapshot(snapshot) else "fallback_rule"


def check_drift(**ctx: Any) -> str:
    """Branch: trigger retraining when PSI on key features exceeds 0.2."""
    from dropout.pipeline_tasks import max_feature_psi

    psi = max_feature_psi(ctx["ti"].xcom_pull(task_ids="build_features"))
    return "trigger_retraining" if psi > 0.2 else "no_drift"
```

dags/dropout_training_dag.py – weekly scoring DAG, tasks and dependencies

```
with DAG(
    dag_id="dropout_weekly_scoring",
    description="Weekly risk scoring and tier assignment",
    schedule="0 2 * * 0",  # Sunday 02:00 (Asia/Ho_Chi_Minh)
    start_date=pendulum.datetime(2026, 1, 4, tz="Asia/Ho_Chi_Minh"),
    catchup=False,
    max_active_runs=1,
    default_args=DEFAULT_ARGS,
    tags=["dropout", "scoring"],
) as scoring_dag:
    t_extract = PythonOperator(
```

```
        task_id="extract_data", python_callable=extract_data, sla=timedelta(hours=1)
    )
    t_validate = BranchPythonOperator(task_id="validate_data", python_callable=validate_data)
    t_features = PythonOperator(task_id="build_features", python_callable=build_features)
    t_score = PythonOperator(task_id="score_students", python_callable=score_students)
    t_fallback = PythonOperator(task_id="fallback_rule", python_callable=fallback_rule)
    t_assign = PythonOperator(
        task_id="assign_tiers",
        python_callable=assign_tiers,
        trigger_rule=TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS,  # after ML or fallback path
        sla=timedelta(hours=4),  # ready before Monday 06:00
    )
    t_drift = BranchPythonOperator(task_id="check_drift", python_callable=check_drift)
    t_retrain = TriggerDagRunOperator(
        task_id="trigger_retraining", trigger_dag_id="dropout_training"
    )
    t_no_drift = EmptyOperator(task_id="no_drift")

    t_extract >> t_validate >> [t_features, t_fallback]
    t_features >> t_score >> t_assign
    t_fallback >> t_assign
    t_features >> t_drift >> [t_retrain, t_no_drift]
```

dags/dropout_training_dag.py – training DAG

```
def run_matrix(**_: Any) -> None:
    """Run the experiment matrix and log every configuration to MLflow."""
    from dropout.experiments import main

    main()


with DAG(
    dag_id="dropout_training",
    description="Retrain, evaluate, gate and register the dropout model",
    schedule="0 3 1 1,5,9 *",  # ~2 weeks after each of the 3 semesters ends
    start_date=pendulum.datetime(2026, 1, 1, tz="Asia/Ho_Chi_Minh"),
    catchup=False,
    max_active_runs=1,
    default_args=DEFAULT_ARGS,
    tags=["dropout", "training"],
) as training_dag:
    t_snapshot = PythonOperator(task_id="snapshot_labelled_data", python_callable=extract_data)
    t_validate_tr = PythonOperator(
        task_id="validate_training_data", python_callable=validate_training_data
    )
    t_train = PythonOperator(
        task_id="train_and_track_experiments",
        python_callable=run_matrix,
        execution_timeout=timedelta(hours=3),
    )
    t_wait_approval = EmptyOperator(task_id="await_human_approval")  # promotion is manual

    t_snapshot >> t_validate_tr >> t_train >> t_wait_approval
```

3.4 Operational considerations

- **Failure handling.** Two retries with exponential back-off starting at 10 minutes; 1-hour execution timeout per task (3 hours for training). Invalid data does not fail the DAG but branches to the fallback rule. `max_active_runs=1` prevents overlapping runs; dated inputs/outputs make every task idempotent. Validation failure in the training DAG stops the run, so the current champion stays in place.
- **Alerting.** `on_failure_callback` e-mails the data team with the task and log link. SLAs (1 h for extraction, 4 h for lists) raise SLA-miss alerts. Fallback activations and drift-triggered retraining are logged and reported.

- **Monitoring.** Airflow UI (durations, success rate); MLflow (metrics per model version); weekly PSI and risk-score distribution (Evidently/Grafana, as in Assignment 1); leading indicators such as the share of Tier A contacted within 7 days. When semester labels arrive, real PR-AUC and Recall@k are recomputed and compared with the validation estimates.

4. Code Quality & Documentation 20%

4.1 Code standards

- **Code style:** PEP 8, enforced and formatted by `ruff` (line length 100; rule sets E, F, I, B, UP, D). Result: `All checks passed!`
- **Docstrings:** Google convention on every public module, class and function, enforced by ruff's D rules.
- **Type hints:** complete on parameters and return values; `dataclass` for experiment configs, results and settings.
- **Comments:** only where intent is not obvious (e.g. why CV is computed, why a column name is stripped).
- **Tests:** 6 pytest unit tests (validation rejects bad ranges, reproducible disjoint split, no sensitive features, zero-enrolment safety, top-k metrics, PSI detects shift): `6 passed`. A README documents layout and commands.

`pyproject.toml – enforced code standards`

```
[tool.ruff]
line-length = 100
target-version = "py311"

[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP", "D"]
ignore = ["D100", "D104", "D107"]

[tool.ruff.lint.pydocstyle]
convention = "google"
```

`src/dropout/evaluate.py – style, type hints and docstrings`

```
def top_k_mask(scores: np.ndarray, fraction: float) -> np.ndarray:
    """Boolean mask selecting the ``fraction`` highest-risk students."""
    k = max(1, int(round(len(scores) * fraction)))
    order = np.argsort(-scores, kind="stable")
    mask = np.zeros(len(scores), dtype=bool)
    mask[order[:k]] = True
    return mask


def recall_at(y: np.ndarray, scores: np.ndarray, fraction: float) -> float:
    """Share of all dropouts that fall inside the top ``fraction`` list."""
    return float(y[top_k_mask(scores, fraction)].sum() / max(1, y.sum()))
```

4.2 Configuration

Two kinds of configuration are separated. `params.yaml` holds every value that affects results (seed, split ratios, feature groups, tier sizes, gates, selection rule) and is version-controlled, so a run is fully described by *git commit + data hash + params.yaml*. Environment variables hold infrastructure locations and secrets (tracking URI, data path, alert address); they are never committed, and only `.env.example` is.

`params.yaml – configuration file (excerpt)`

```
project:
  name: dropout-early-warning
  seed: 42

data:
  raw_path: data/raw/students.csv
  separator: ";"
  target_column: Target
  positive_label: Dropout          # label 1 = student drops out
  test_size: 0.20                  # final hold-out, used once for the champion
  val_size: 0.20                   # used for model selection
```

```
  max_missing_ratio: 0.05          # validation gate
  min_rows: 1000

features:
  sensitive: [Gender, Nacionality, Marital status, International]
  ...

evaluation:
  tier_a_fraction: 0.10            # advisors contact the top 10 %
  tier_b_fraction: 0.25            # care team covers up to the top 25 %
  primary_metric: pr_auc
  cv_folds: 5

selection:
  # From Assignment 1, trade-off 5.1: a complex model must beat the simple,
  # explainable baseline by a clear margin, otherwise keep the baseline.
  simple_baseline: E02
  min_gain_over_baseline: 0.02

quality_gate:
  min_pr_auc: 0.80                 # proxy dataset (base rate ~32 %)
  max_fairness_gap: 0.10           # recall gap between groups at top 25 %
  max_regression_vs_champion: 0.01

mlflow:
  experiment_name: dropout-early-warning
  registered_model: dropout-risk
```

src/dropout/config.py – environment variables and config loading

```
def load_settings() -> Settings:
    """Read environment variables, falling back to safe local defaults.

    Returns:
        Settings: resolved infrastructure settings.
    """
    data_env = os.getenv("DROPOUT_DATA_PATH")
    return Settings(
        tracking_uri=os.getenv("MLFLOW_TRACKING_URI", f"sqlite:///{PROJECT_ROOT / 'mlflow.db'}"),
        params_path=Path(os.getenv("DROPOUT_PARAMS_PATH", PROJECT_ROOT / "params.yaml")),
        data_path=Path(data_env) if data_env else None,
    )


def load_params(path: str | Path | None = None) -> dict[str, Any]:
    """Load the YAML parameter file.

    Args:
        path: Optional explicit path; defaults to ``DROPOUT_PARAMS_PATH`` or
            ``params.yaml`` at the project root.

    Returns:
        dict: nested configuration dictionary.
    """
    params_path = Path(path) if path else load_settings().params_path
    with open(params_path, encoding="utf-8") as fh:
        params: dict[str, Any] = yaml.safe_load(fh)
    return params
```

5. Reproducibility & Versioning Strategy 10%

Asset	Strategy
Code	GitHub flow: `main` is always releasable; work happens on `feat/…` or `fix/…` branches merged through pull requests after CI (ruff + pytest) passes and one review. Releases are annotated semantic-version tags (`v0.2.0`); the commit hash is stored on every registered model and the tag on the Docker image.
Data	Each extraction is an immutable, dated snapshot; its SHA-256 is logged on every MLflow run and model version. In production, DVC with a MinIO remote tracks snapshots: the small `.dvc` pointer is committed with the code, so checking out a commit restores the exact data.
Model	MLflow Model Registry, registered model `dropout-risk`; versions increment automatically. Aliases: `challenger` (passed the gate), `champion` (serving), `previous-champion` (rollback). Mandatory tags: `exp_id`, `git_commit`, `data_sha256`, `approved_by`. Run naming: `<exp_id>-<algorithm>`, e.g. `E02-logreg`.
Environment	Exact versions pinned in `requirements.txt`; Docker base image pinned to `python:3.11.10-slim`; `PYTHONHASHSEED` fixed.

Code and data versioning

```
# GitHub flow: feature branch -> pull request (CI: ruff + pytest) -> main -> tag
git switch -c feat/early-warning-features
git commit -m "Add approval-rate and grade-trend features"
git push -u origin feat/early-warning-features      # open PR; CI must pass
git switch main && git pull
git tag -a v0.2.0 -m "Experiment matrix E01-E12, LR champion"
git push origin v0.2.0

# Data versioning with DVC (production), remote = MinIO
dvc add data/raw/students.csv
git add data/raw/students.csv.dvc .gitignore
git commit -m "Data snapshot 2026-S3" && dvc push

# Every registered model carries the commit and data hash (registry.py / experiments.py)
#   tags: exp_id=E02, git_commit=bbc0168, data_sha256=19737f02127e, approved_by=...
```

src/dropout/config.py – random seed settings

```
def set_global_seed(seed: int) -> None:
    """Fix every source of randomness used by the pipeline.

    Args:
        seed: integer seed taken from ``params.yaml``.
    """
    random.seed(seed)
    np.random.seed(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)

# ...and the same seed is passed explicitly everywhere randomness exists:
#   train_test_split(..., random_state=seed)      data.py
#   StratifiedKFold(5, shuffle=True, random_state=seed), RandomizedSearchCV(random_state=seed)
#   LogisticRegression / RandomForest / XGBClassifier(random_state=seed), SMOTE(random_state=seed)
```

Dockerfile

```
# Reproducible training/scoring image for the dropout early-warning pipeline.
FROM python:3.11.10-slim

# Deterministic Python behaviour and no .pyc clutter
ENV PYTHONHASHSEED=42 \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PYTHONPATH=/app/src
```

```
WORKDIR /app

# Install pinned dependencies first to maximise layer caching
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy code and configuration (data is mounted or pulled with DVC at runtime)
COPY src/ src/
COPY params.yaml .

# Run as non-root
RUN useradd --create-home appuser
USER appuser

# Default: run the experiment matrix; Airflow overrides the command per task
CMD ["python", "-m", "dropout.experiments"]
```

Evidence. Re-running all twelve experiments from an empty MLflow store reproduced every metric exactly (only wall-clock time differs), and a unit test asserts that two calls to the split function return identical partitions.

6. Limitations and Next Steps

- **Proxy data.** The UCI dataset (Portugal) has a 32 % dropout rate, unlike the assumed 2.7 % per semester at BTEC, and lacks weekly features. Results demonstrate the pipeline and evaluation method, not expected production performance.
- **Random split.** Without enrolment dates, the stratified split may be optimistic compared with a time-based split.
- **Not yet run end-to-end in production infrastructure.** DAGs load in Airflow and all task functions were executed end-to-end (4,424 students scored; 442 assigned to Tier A, 664 to Tier B), but not under a live scheduler; the Docker image was not built in this environment.
- **Next steps.** Connect real exports; switch to semester-based splitting; add weekly attendance and LMS features; re-run the E02 vs. E07 comparison under the 0.02 rule.

References

1. Realinho, V., Machado, J., Baptista, L., & Martins, M. V. (2022). Predicting Student Dropout and Academic Success. *Data*, 7(11), 146. Dataset: UCI Machine Learning Repository, id 697.
2. Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly Media.
3. Kästner, C. (2025). *Machine Learning in Production: From Models to Products*. MIT Press.
4. Sculley, D. et al. (2015). Hidden Technical Debt in Machine Learning Systems. *NeurIPS*.
5. MLflow documentation: Tracking and Model Registry (aliases). Apache Airflow 2.10 documentation: DAGs, operators, trigger rules.
6. Nguyen Thi Hong Hanh (2026). DDM501 Assignment 1: Early Warning System for Students at Risk of Dropping Out.